

CSS Practicals with Answers

Web Technology | Diploma Engineering
Gujarat Technological University

Covers: Selectors · Box Model · Colors · Fonts · Flexbox · Grid · Transitions · Animations · Responsive Design · Pseudo-classes

Practical 1: CSS Selectors

Aim: Demonstrate the use of various CSS selectors (element, class, id, universal, attribute).

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Selectors</title>
  <style>
    /* Universal Selector */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    /* Element Selector */
    p {
      font-size: 16px;
      color: #333;
    }

    /* Class Selector */
    .highlight {
      background-color: yellow;
      font-weight: bold;
    }

    /* ID Selector */
    #main-title {
      color: navy;
      font-size: 24px;
      text-align: center;
    }

    /* Attribute Selector */
    input[type="text"] {
      border: 2px solid blue;
      padding: 5px;
    }

    /* Descendant Selector */
    div p {
      color: green;
    }

    /* Group Selector */
    h1, h2, h3 {
      font-family: Arial, sans-serif;
    }
  </style>
</head>
<body>
  <h1 id="main-title">CSS Selectors Demo</h1>
```

```

<p>This is a normal paragraph.</p>
<p class="highlight">This paragraph is highlighted using class selector.</p>
<div>
  <p>This paragraph is inside a div (green via descendant selector).</p>
</div>
<input type="text" placeholder="Text input styled via attribute selector">
</body>
</html>

```

Explanation:

- ***** (Universal) — applies to all elements. **p** (Element) — targets all `<p>` tags. **.highlight** (Class) — targets elements with `class='highlight'`. **#main-title** (ID) — targets unique element. **input[type='text']** (Attribute) — targets specific attribute values. **div p** (Descendant) — targets `<p>` inside `<div>`.

Output/Result: Each selector applies its respective style. The paragraph inside div appears green; the highlighted paragraph has a yellow background; the text input has a blue border.

Practical 2: CSS Box Model

Aim: Understand and demonstrate the CSS Box Model (margin, border, padding, content).

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>Box Model</title>
  <style>
    .box {
      width: 200px;
      height: 100px;
      background-color: lightblue;

      /* Content area: 200x100 px */
      padding: 20px; /* Space inside border */
      border: 5px solid navy; /* Border around padding */
      margin: 30px; /* Space outside border */
    }

    .box-sizing-demo {
      width: 200px;
      height: 100px;
      padding: 20px;
      border: 5px solid red;
      box-sizing: border-box; /* Padding & border included in width */
      background-color: lightyellow;
      margin: 30px;
    }
  </style>
</head>
<body>
  <h2>Default Box Model</h2>
  <div class="box">
    Content Area
  </div>

  <h2>box-sizing: border-box</h2>

```

```

<div class="box-sizing-demo">
  Content with border-box
</div>
</body>
</html>

```

Explanation:

The CSS Box Model wraps every HTML element: **Content** (actual content) → **Padding** (space between content and border) → **Border** (surrounds padding) → **Margin** (space outside border). With default box model, total width = content + padding + border. With **box-sizing: border-box**, width includes padding and border, making layout calculations easier.

Output/Result: First box total width = 200+40+10 = 250px. Second box stays 200px wide because border-box includes padding & border in width.

Practical 3: CSS Colors and Backgrounds

Aim: Apply different color formats and background properties in CSS.

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>Colors and Backgrounds</title>
  <style>
    .color-named    { color: red; background-color: lightyellow; padding: 10px; }
    .color-hex      { color: #2d6a4f; background-color: #b7e4c7; padding: 10px; }
    .color-rgb      { color: rgb(180, 50, 20); background-color: rgb(255,220,200); padding: 10px; }
    .color-rgba     { background-color: rgba(0, 0, 255, 0.2); padding: 10px; }
    .color-hsl      { color: hsl(270, 60%, 40%); padding: 10px; }

    .bg-image {
      background-image: url('https://via.placeholder.com/300x100');
      background-repeat: no-repeat;
      background-size: cover;
      background-position: center;
      height: 100px;
      color: white;
      display: flex;
      align-items: center;
      justify-content: center;
    }

    .bg-gradient {
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white;
      padding: 20px;
      text-align: center;
    }
  </style>
</head>
<body>
  <p class="color-named">Named Color: red text on lightyellow</p>
  <p class="color-hex">HEX Color: #2d6a4f on #b7e4c7</p>
  <p class="color-rgb">RGB Color: rgb(180,50,20)</p>
  <p class="color-rgba">RGBA: semi-transparent blue background</p>

```

```

    <p class="color-hsl">HSL Color: hsl(270, 60%, 40%)</p>
    <div class="bg-gradient">Linear Gradient Background</div>
</body>
</html>

```

Explanation:

CSS supports colors in multiple formats: **Named** (red, blue), **HEX** (#RRGGBB), **RGB** (0-255), **RGBA** (with alpha/opacity), **HSL** (hue, saturation, lightness). Backgrounds can be solid colors, images (**background-image**), or gradients (**linear-gradient**).

Output/Result: Each paragraph shows a different color format. The gradient div shows a smooth purple-to-violet background.

Practical 4: CSS Typography (Fonts & Text)

Aim: Apply CSS font properties and text styling to web content.

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>Typography</title>
  <!-- Import Google Font -->
  <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
  <style>
    body {
      font-family: 'Roboto', Arial, sans-serif;
      font-size: 16px;
      line-height: 1.6;
      color: #333;
    }

    h1 {
      font-size: 2em;          /* 32px */
      font-weight: bold;
      letter-spacing: 2px;
      text-transform: uppercase;
      color: #1a237e;
    }

    .italic-text    { font-style: italic; }
    .underline      { text-decoration: underline; }
    .strikethrough  { text-decoration: line-through; }
    .overline       { text-decoration: overline; }
    .center         { text-align: center; }
    .justify        { text-align: justify; width: 400px; }
    .shadow         { text-shadow: 2px 2px 4px rgba(0,0,0,0.4); font-size: 20px; }
    .indent         { text-indent: 40px; }
    .spaced         { word-spacing: 10px; }
  </style>
</head>
<body>
  <h1>Typography Demo</h1>
  <p class="italic-text">Italic text using font-style: italic</p>
  <p class="underline">Underlined text</p>
  <p class="strikethrough">Strikethrough text</p>

```

```
<p class="center">Center aligned text</p>
<p class="shadow">Text with shadow effect</p>
<p class="indent">This paragraph has a text-indent of 40px applied.</p>
<p class="spaced">Extra word spacing applied here</p>
</body>
</html>
```

Key Properties:

font-family: typeface name. **font-size:** size (px, em, rem, %). **font-weight:** normal/bold/100-900. **font-style:** normal/italic/oblique. **text-align:** left/right/center/justify. **text-decoration:** underline/overline/line-through. **text-shadow:** horizontal vertical blur color. **letter-spacing** and **word-spacing** control character/word gaps.

Output/Result: Each paragraph demonstrates a distinct typographic property. The heading appears uppercase with letter-spacing.

Practical 5: CSS Display and Positioning

Aim: Demonstrate CSS display values and positioning (static, relative, absolute, fixed, sticky).

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Display and Position</title>
  <style>
    /* Display types */
    .block-el { display: block; background: #cfe2ff; padding: 8px; margin: 5px 0; }
    .inline-el { display: inline; background: #d1e7dd; padding: 4px; margin: 2px; }
    .inline-blk { display: inline-block; width: 100px; height: 50px;
                  background: #fff3cd; text-align: center; line-height: 50px; }

    /* Positioning */
    .container {
      position: relative;
      height: 150px;
      background: #f8d7da;
      border: 2px dashed #842029;
      margin: 20px 0;
    }

    .relative-box {
      position: relative;
      top: 10px;
      left: 20px;
      background: #0d6efd;
      color: white;
      padding: 5px 10px;
      width: fit-content;
    }

    .absolute-box {
      position: absolute;
      bottom: 10px;
      right: 10px;
      background: #198754;
      color: white;
```

```

        padding: 5px 10px;
    }

    .fixed-box {
        position: fixed;
        top: 10px;
        right: 10px;
        background: #dc3545;
        color: white;
        padding: 5px 10px;
        font-size: 12px;
        border-radius: 4px;
    }
</style>
</head>
<body>
    <div class="fixed-box">Fixed (top-right)</div>

    <div class="block-el">Block element – full width</div>
    <span class="inline-el">Inline A</span>
    <span class="inline-el">Inline B</span>
    <div class="inline-blk">Inline-Block 1</div>
    <div class="inline-blk">Inline-Block 2</div>

    <div class="container">
        Parent (position: relative)
        <div class="relative-box">Relative: top 10, left 20</div>
        <div class="absolute-box">Absolute: bottom-right</div>
    </div>
</body>
</html>

```

Explanation:

static: default, follows document flow. **relative:** offset from its normal position, space preserved. **absolute:** positioned relative to nearest positioned ancestor, removed from flow. **fixed:** positioned relative to viewport, stays during scroll. **sticky:** relative until scroll threshold, then fixed. **z-index** controls stack order for overlapping elements.

Output/Result: Fixed box stays at top-right during scroll. Absolute box sits at bottom-right of parent. Relative box shifts 10px down, 20px right from normal.

Practical 6: CSS Flexbox Layout

Aim: Build flexible layouts using CSS Flexbox properties.

Code:

```

<!DOCTYPE html>
<html>
<head>
    <title>Flexbox Layout</title>
    <style>
        .flex-container {
            display: flex;
            flex-direction: row;          /* row | column | row-reverse */
            flex-wrap: wrap;              /* wrap items to next line */
            justify-content: space-between; /* main axis alignment */

```

```

    align-items: center;          /* cross axis alignment */
    gap: 10px;
    background: #e9ecef;
    padding: 15px;
    border-radius: 8px;
}

.flex-item {
    background: #0d6efd;
    color: white;
    padding: 20px;
    border-radius: 6px;
    font-weight: bold;
    flex: 1 1 100px; /* grow shrink basis */
    text-align: center;
}

.flex-item:nth-child(2) { flex: 2; background: #6610f2; }
.flex-item:nth-child(3) { align-self: flex-end; background: #198754; }

/* Navigation bar using flexbox */
nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #212529;
    padding: 12px 20px;
    border-radius: 8px;
    margin-top: 20px;
}
nav .logo { color: gold; font-weight: bold; font-size: 18px; }
nav ul     { display: flex; list-style: none; gap: 20px; margin: 0; padding: 0; }
nav ul li a { color: white; text-decoration: none; }
</style>
</head>
<body>
<h2>Flex Container</h2>
<div class="flex-container">
    <div class="flex-item">Item 1</div>
    <div class="flex-item">Item 2 (flex: 2)</div>
    <div class="flex-item">Item 3 (align-self: end)</div>
    <div class="flex-item">Item 4</div>
</div>

<nav>
    <div class="logo">MyBrand</div>
    <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
    </ul>
</nav>
</body>
</html>

```

Key Flexbox Properties:

display: flex: activates flexbox on container. **flex-direction:** row/column. **justify-content:** flex-start/center/space-between/space-around (main axis). **align-items:** flex-start/center/flex-end/stretch (cross axis). **flex-wrap:** wrap/nowrap. **gap:** space between items. **flex: grow shrink basis** on child items. **align-self:** overrides align-items per item.

Output/Result: Items are distributed with space-between. Item 2 is twice as wide. Item 3 sits at the bottom (flex-end). Nav bar has logo and links on opposite sides.

Practical 7: CSS Grid Layout

Aim: Create two-dimensional layouts using CSS Grid.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Grid</title>
  <style>
    .grid-container {
      display: grid;
      grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
      grid-template-rows: auto;
      gap: 10px;
      padding: 15px;
      background: #f8f9fa;
      border-radius: 8px;
    }

    .grid-item {
      background: #0d6efd;
      color: white;
      padding: 20px;
      text-align: center;
      border-radius: 6px;
      font-weight: bold;
    }

    /* Span columns/rows */
    .span-2 { grid-column: span 2; background: #6610f2; }
    .span-row { grid-row: span 2; background: #198754; }

    /* Page layout using grid areas */
    .page {
      display: grid;
      grid-template-areas:
        "header header header"
        "sidebar content content"
        "footer footer footer";
      grid-template-columns: 200px 1fr 1fr;
      grid-template-rows: 60px 200px 50px;
      gap: 8px;
      margin-top: 20px;
    }

    .page header { grid-area: header; background: #212529; color: white; display: flex; align-items: center; justify-content: space-between; padding: 5px 10px; }
    .page aside { grid-area: sidebar; background: #e9ecef; padding: 12px; border-radius: 6px; }
    .page main { grid-area: content; padding: 10px; }
    .page footer { grid-area: footer; background: #6c757d; color: white; text-align: center; padding: 5px 10px; }
```

```

        .page main      { grid-area: content; background: #ffffff; padding: 12px; border-radius:6px; border:1px solid #ccc; }
        .page footer    { grid-area: footer;  background: #343a40; color: white; display:flex; align-items:center; justify-content:center; }
    </style>
</head>
<body>
    <h2>Basic Grid</h2>
    <div class="grid-container">
        <div class="grid-item">1</div>
        <div class="grid-item span-2">2 (spans 2 cols)</div>
        <div class="grid-item span-row">3 (spans 2 rows)</div>
        <div class="grid-item">4</div>
        <div class="grid-item">5</div>
        <div class="grid-item">6</div>
    </div>

    <h2>Page Layout with Grid Areas</h2>
    <div class="page">
        <header>Header</header>
        <aside>Sidebar</aside>
        <main>Main Content Area</main>
        <footer>Footer</footer>
    </div>
</body>
</html>

```

Key Grid Properties:

display: grid: activates grid. **grid-template-columns:** define column sizes (px, %, fr, repeat). **grid-template-rows:** define row sizes. **gap:** gutters between cells. **grid-column: span N:** item spans N columns. **grid-row: span N:** item spans N rows. **grid-template-areas:** named layout regions. **grid-area:** assign item to named area.

Output/Result: Item 2 spans 2 columns (purple). Item 3 spans 2 rows (green). Page layout shows classic header/sidebar/content/footer structure using named areas.

Practical 8: CSS Pseudo-classes and Pseudo-elements

Aim: Use CSS pseudo-classes (:hover, :focus, :nth-child) and pseudo-elements (::before, ::after).

Code:

```

<!DOCTYPE html>
<html>
<head>
    <title>Pseudo-classes & Pseudo-elements</title>
    <style>
        /* :hover – on mouse over */
        .btn {
            background: #0d6efd;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            transition: background 0.3s;
        }
        .btn:hover { background: #0a58ca; transform: scale(1.05); }
    </style>
</head>
<body>
    <div>
        <button class="btn">Click Me!</button>
    </div>
</body>
</html>

```

```

/* :focus – when input is focused */
input {
  border: 2px solid #ccc;
  padding: 8px;
  border-radius: 4px;
  outline: none;
  transition: border-color 0.3s;
}
input:focus { border-color: #0d6efd; box-shadow: 0 0 4px rgba(13,110,253,0.4); }

/* :nth-child – select specific children */
li:nth-child(odd) { background: #e9ecef; }
li:nth-child(even) { background: #dee2e6; }
li:first-child { color: #0d6efd; font-weight: bold; }
li:last-child { color: #dc3545; font-weight: bold; }

/* :not – exclude elements */
p:not(.special) { color: gray; }
p.special { color: #198754; font-weight: bold; }

/* ::before and ::after pseudo-elements */
.quote {
  position: relative;
  padding: 10px 20px;
  background: #f8f9fa;
  border-left: 4px solid #0d6efd;
  margin: 10px 0;
}
.quote::before {
  content: '"';
  font-size: 60px;
  color: #0d6efd;
  position: absolute;
  top: -10px;
  left: 8px;
  line-height: 1;
  opacity: 0.3;
}

/* ::first-letter, ::first-line */
.dropcap::first-letter {
  font-size: 3em;
  float: left;
  margin-right: 4px;
  color: #6610f2;
  font-weight: bold;
  line-height: 0.8;
}
</style>
</head>
<body>
  <button class="btn">Hover Me</button><br><br>
  <input type="text" placeholder="Focus me..."><br><br>

<ul>

```

```

    <li>First Item</li>
    <li>Second Item</li>
    <li>Third Item</li>
    <li>Fourth Item</li>
    <li>Last Item</li>
</ul>

<p>Normal paragraph (gray via :not(.special))</p>
<p class="special">Special paragraph (green)</p>

<div class="quote">CSS is the language we use to style a web page.</div>

<p class="dropcap">This paragraph has a large decorative first letter created
using the ::first-letter pseudo-element in CSS.</p>
</body>
</html>

```

Explanation:

Pseudo-classes apply styles based on state or position: :hover, :focus, :active, :visited, :nth-child(n), :first-child, :last-child, :not(selector). **Pseudo-elements** style a specific part of an element: ::before (inserts content before), ::after (inserts content after), ::first-letter, ::first-line, ::placeholder, ::selection.

Output/Result: Button changes color on hover. Input glows blue on focus. List items alternate background colors. Quote has a large decorative quotation mark via ::before.

Practical 9: CSS Transitions and Animations

Aim: Create smooth transitions and keyframe animations using CSS.

Code:

```

<!DOCTYPE html>
<html>
<head>
  <title>Transitions and Animations</title>
  <style>
    /* Transitions */
    .transition-box {
      width: 150px;
      height: 150px;
      background: #0d6efd;
      border-radius: 10px;
      transition: all 0.4s ease-in-out; /* property duration timing */
      margin: 10px;
      display: inline-block;
    }
    .transition-box:hover {
      background: #dc3545;
      border-radius: 50%; /* circle on hover */
      transform: rotate(45deg) scale(1.2);
    }

    /* Keyframe Animation */
    @keyframes slideIn {
      0% { transform: translateX(-100%); opacity: 0; }
      60% { transform: translateX(10px); opacity: 0.8; }
      100% { transform: translateX(0); opacity: 1; }
    }
  </style>
</head>
</html>

```

```

}

@keyframes pulse {
  0%, 100% { transform: scale(1); background: #198754; }
  50%      { transform: scale(1.15); background: #20c997; }
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

.slide-box {
  animation: slideIn 0.8s ease forwards;
  background: #6610f2; color: white;
  padding: 12px 20px; border-radius: 6px;
  margin: 10px 0; display: inline-block;
}

.pulse-box {
  width: 100px; height: 100px;
  background: #198754; border-radius: 50%;
  animation: pulse 1.5s ease-in-out infinite;
  display: inline-flex; align-items: center;
  justify-content: center; color: white; font-weight: bold;
}

.spinner {
  width: 50px; height: 50px;
  border: 5px solid #e9ecef;
  border-top-color: #0d6efd;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
  display: inline-block; margin: 10px;
}
</style>
</head>
<body>
  <h2>Hover Transition</h2>
  <div class="transition-box"></div>

  <h2>Keyframe Animations</h2>
  <div class="slide-box">Slide In Animation</div><br>
  <div class="pulse-box">Pulse</div>
  <div class="spinner"></div>
</body>
</html>

```

Explanation:

transition: property | duration | timing-function | delay. Smooth change on state change (e.g., :hover).
@keyframes: defines animation steps with from/to or percentages. **animation:** name | duration | timing | delay | iteration | direction | fill-mode. Common timing functions: ease, linear, ease-in, ease-out, ease-in-out, cubic-bezier().

Output/Result: Box morphs into a rotated red circle on hover. Slide-in box animates from left on page load. Pulse circle grows and changes color. Spinner rotates infinitely.

Practical 10: Responsive Web Design with Media Queries

Aim: Build a responsive layout using CSS media queries and the mobile-first approach.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Responsive Design</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    /* ■■ Base (Mobile First) ■■ */
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body { font-family: Arial, sans-serif; font-size: 16px; }

    header {
      background: #212529;
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    nav ul {
      list-style: none;
      display: flex;
      flex-direction: column; /* stacked on mobile */
      gap: 5px;
    }
    nav ul li a { color: white; text-decoration: none; font-size: 14px; }

    .container {
      padding: 15px;
      display: grid;
      grid-template-columns: 1fr; /* 1 column on mobile */
      gap: 15px;
    }

    .card {
      background: white;
      border: 1px solid #dee2e6;
      border-radius: 8px;
      padding: 15px;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    }

    img { width: 100%; height: auto; } /* fluid images */

    /* ■■ Tablet (≥768px) ■■ */
    @media (min-width: 768px) {
      nav ul { flex-direction: row; gap: 20px; }
      .container { grid-template-columns: repeat(2, 1fr); } /* 2 cols */
    }
  </style>
</head>
</html>
```

```

/* ■■ Desktop (≥1024px) ■■ */
@media (min-width: 1024px) {
  .container { grid-template-columns: repeat(3, 1fr); } /* 3 cols */
  body { font-size: 18px; }
}

/* ■■ Print media ■■ */
@media print {
  header, nav { display: none; }
  body { font-size: 12pt; color: black; }
}

/* ■■ Dark mode preference ■■ */
@media (prefers-color-scheme: dark) {
  body { background: #121212; color: #e0e0e0; }
  .card { background: #1e1e1e; border-color: #333; }
}
</style>
</head>
<body>
  <header>
    <div>MyWebsite</div>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Services</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>
  </header>

  <div class="container">
    <div class="card"><h3>Card 1</h3><p>Content here.</p></div>
    <div class="card"><h3>Card 2</h3><p>Content here.</p></div>
    <div class="card"><h3>Card 3</h3><p>Content here.</p></div>
    <div class="card"><h3>Card 4</h3><p>Content here.</p></div>
    <div class="card"><h3>Card 5</h3><p>Content here.</p></div>
    <div class="card"><h3>Card 6</h3><p>Content here.</p></div>
  </div>
</body>
</html>

```

Explanation:

Mobile-First: write base styles for small screens, then use min-width media queries to enhance. **@media (min-width: Xpx):** applies styles when viewport $\geq$ X. Common breakpoints: 576px (sm), 768px (md/tablet), 1024px (lg/desktop), 1280px (xl). **viewport meta tag:** required for correct mobile rendering. **Fluid images:** width:100% prevents overflow. **Grid columns** expand from 1→2→3 as screen widens.

Output/Result: On mobile: 1-column layout, stacked nav. On tablet (768px+): 2-column grid, horizontal nav. On desktop (1024px+): 3-column grid. Dark mode applies automatically based on OS preference.

Practical 11: CSS Variables (Custom Properties)

Aim: Use CSS custom properties (variables) for consistent theming and easy maintenance.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Variables</title>
  <style>
    /* Define variables on :root (global scope) */
    :root {
      --primary:    #0d6efd;
      --secondary:  #6610f2;
      --success:    #198754;
      --danger:     #dc3545;
      --bg:         #f8f9fa;
      --text:       #212529;
      --radius:     8px;
      --shadow:      0 2px 8px rgba(0,0,0,0.12);
      --transition: 0.3s ease;
      --font-size-base: 16px;
    }

    /* Dark theme overrides */
    [data-theme="dark"] {
      --bg:    #121212;
      --text:  #e0e0e0;
      --primary: #4dabf7;
    }

    body {
      background: var(--bg);
      color: var(--text);
      font-size: var(--font-size-base);
      font-family: Arial, sans-serif;
      padding: 20px;
      transition: background var(--transition), color var(--transition);
    }

    .btn {
      padding: 10px 20px;
      border: none;
      border-radius: var(--radius);
      cursor: pointer;
      transition: opacity var(--transition);
      font-size: var(--font-size-base);
    }
    .btn:hover { opacity: 0.85; }

    .btn-primary { background: var(--primary); color: white; }
    .btn-secondary { background: var(--secondary); color: white; }
    .btn-success { background: var(--success); color: white; }
    .btn-danger { background: var(--danger); color: white; }

    .card {
      background: white;
      border-radius: var(--radius);
      box-shadow: var(--shadow);
    }
  </style>
</head>
</html>
```



```

padding: 20px;
margin: 15px 0;
border-left: 4px solid var(--primary);
}

/* calc() with variables */
.wide-btn {
width: calc(100% - 40px);
background: var(--secondary);
color: white;
padding: 12px;
border-radius: var(--radius);
text-align: center;
margin-top: 10px;
}
</style>
</head>
<body>
<h2>CSS Variables Demo</h2>
<div>
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
</div>
<div class="card">
<h3>Card using CSS Variables</h3>
<p>Border-left uses --primary, shadow uses --shadow, radius uses --radius.</p>
</div>
<div class="wide-btn">Wide button using calc() + variables</div>
</body>
</html>

```

Explanation:

--variable-name: value: defines a custom property. **var(--variable-name):** uses the variable. Variables defined on **:root** are globally available. Variables are scoped — can be overridden for specific selectors (e.g., dark theme via `[data-theme='dark']`). **calc()** can combine variables with arithmetic. Variables cascade and inherit like other CSS properties.

Output/Result: All buttons share `--radius` and `--transition`. Changing `--primary` in `:root` updates all elements using it instantly. Dark theme toggle changes `--bg` and `--text` globally.

Practical 12: CSS Card & Navigation UI Component

Aim: Build a complete styled card component and responsive navigation bar using CSS.

Code:

```

<!DOCTYPE html>
<html>
<head>
<title>UI Components</title>
<style>
:root {
--primary: #0d6efd;
--dark: #212529;
--light: #f8f9fa;

```

```

    --radius: 10px;
}

* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: 'Segoe UI', sans-serif; background: #eef2ff; padding: 20px; }

/* ■■ Navbar ■■ */
.navbar {
    background: var(--dark);
    padding: 0 24px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    height: 60px;
    border-radius: var(--radius);
    margin-bottom: 30px;
}
.navbar .brand { color: gold; font-size: 20px; font-weight: 700; letter-spacing: 1px; }
.navbar ul { display: flex; list-style: none; gap: 24px; }
.navbar a { color: #adb5bd; text-decoration: none; font-size: 14px; transition: color 0.2s; }
.navbar a:hover { color: white; }
.navbar .cta { background: var(--primary); color: white !important;
    padding: 6px 14px; border-radius: 6px; }

/* ■■ Card Grid ■■ */
.cards { display: grid; grid-template-columns: repeat(auto-fill, minmax(220px, 1fr)); gap: 20px; }

.card {
    background: white;
    border-radius: var(--radius);
    overflow: hidden;
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
    transition: transform 0.25s, box-shadow 0.25s;
}
.card:hover { transform: translateY(-4px); box-shadow: 0 8px 24px rgba(0,0,0,0.15); }

.card-img {
    width: 100%; height: 140px;
    background: linear-gradient(135deg, var(--primary), #6610f2);
    display: flex; align-items: center; justify-content: center;
    font-size: 40px;
}
.card-body { padding: 16px; }
.card-title { font-size: 16px; font-weight: 700; margin-bottom: 8px; color: var(--dark); }
.card-text { font-size: 13px; color: #6c757d; line-height: 1.5; }
.card-footer {
    padding: 10px 16px;
    border-top: 1px solid #e9ecef;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.badge {
    background: #e7f1ff;
    color: var(--primary);

```

```

padding: 3px 8px;
border-radius: 20px;
font-size: 12px;
font-weight: 600;
}
.btn-card {
background: var(--primary);
color: white;
border: none;
padding: 6px 14px;
border-radius: 6px;
cursor: pointer;
font-size: 13px;
transition: background 0.2s;
}
.btn-card:hover { background: #0a58ca; }
</style>
</head>
<body>
<nav class="navbar">
  <div class="brand">&#9670; PlacePortal</div>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Jobs</a></li>
    <li><a href="#">Students</a></li>
  </ul>
  <a href="#" class="cta">Login</a>
</nav>

<div class="cards">
  <div class="card">
    <div class="card-img">&#128187;</div>
    <div class="card-body">
      <div class="card-title">Frontend Developer</div>
      <div class="card-text">Build responsive UIs using React, CSS and modern web technologies.</div>
    </div>
    <div class="card-footer">
      <span class="badge">React</span>
      <button class="btn-card">Apply</button>
    </div>
  </div>
  <div class="card">
    <div class="card-img">&#9889;</div>
    <div class="card-body">
      <div class="card-title">Backend Developer</div>
      <div class="card-text">Develop scalable APIs and manage databases using Django & Python.</div>
    </div>
    <div class="card-footer">
      <span class="badge">Django</span>
      <button class="btn-card">Apply</button>
    </div>
  </div>
  <div class="card">
    <div class="card-img">&#128202;</div>
    <div class="card-body">

```

```
<div class="card-title">Data Analyst</div>
<div class="card-text">Analyze data, create visualizations and derive business insights.</div>
</div>
<div class="card-footer">
  <span class="badge">Python</span>
  <button class="btn-card">Apply</button>
</div>
</div>
</div>
</body>
</html>
```

Explanation:

This practical combines multiple CSS concepts: **flexbox** for the navbar layout, **CSS grid with auto-fill/minmax** for a responsive card grid, **CSS variables** for consistent theming, **transitions + transform** for the card hover lift effect, **overflow: hidden** on card to clip the image, and **gradient backgrounds** for the card images.

Output/Result: Dark navbar with gold brand, gray links turning white on hover, and a blue CTA button. Cards lift up 4px on hover with deeper shadow. Grid auto-adjusts columns based on screen width.

End of CSS Practicals — All the best! ■